Artificial Intelligence

A beginner-friendly knowledge document for Retrieval-Augmented Generation (RAG) applications

Purpose of This Document

This document introduces key concepts in Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), Large Language Models (LLMs), embeddings, vector databases, and Retrieval-Augmented Generation (RAG). It is written as a compact reference document that can be uploaded to a RAG application for testing document ingestion, chunking, retrieval, and question answering.

Important: This is an educational overview. It is not a substitute for technical documentation, professional advice, or verification against current library/model documentation.

1. What Is Artificial Intelligence?

Artificial Intelligence is a field of computing concerned with building systems that perform tasks commonly associated with human intelligence. These tasks can include understanding language, recognizing images, making predictions, planning actions, solving problems, and generating content. AI systems may use explicit rules, statistical models, learned representations, or combinations of these approaches.

Common applications of AI include:

- Natural language processing: classifying text, summarizing documents, translating languages, and answering questions.
- Computer vision: identifying objects, classifying images, and analyzing visual scenes.
- Recommendation systems: ranking products, films, courses, or content based on user and item signals.
- Robotics and planning: interpreting sensor data, selecting actions, and controlling physical systems.
- Generative AI: creating text, images, audio, video, or code from prompts and other context.

2. Artificial Intelligence, Machine Learning, and Deep Learning

AI is the broad field. Machine Learning is a subfield of AI in which models learn patterns from data rather than relying only on manually written rules. Deep Learning is a subfield of ML that uses neural networks with multiple layers to learn useful representations from data.

Term	Meaning	Example
Artificial Intelligence	Systems that perform intelligent tasks.	A rule-based planning system.
Machine Learning	Models learn patterns from examples.	Predicting house prices from past sales.
Deep Learning	ML using multi-layer neural networks.	A neural network classifying images.
Generative AI	Models generate new content based on learned patterns.	Drafting a summary or generating code.

3. Large Language Models (LLMs)

A Large Language Model is a machine-learning model trained on large amounts of text and other data to process and generate language. Given a prompt, an LLM predicts and produces a sequence of tokens. Depending on its

training and tools, it may summarize text, answer questions, extract structured information, or assist with code. An LLM can produce incorrect or unsupported statements, so important outputs should be checked.

Common limitations of LLMs

- Hallucination: producing plausible-sounding content that is inaccurate or not supported by evidence.
- Knowledge limitations: model training may not include recent events or private organizational documents.
- Context limits: only a finite amount of text can be processed in a single request.
- Prompt sensitivity: small changes in instructions or context can change the output.
- Security risks: retrieved documents or user prompts may contain malicious instructions or sensitive information.

4. Embeddings and Semantic Search

An embedding is a numerical vector that represents the meaning or characteristics of an item such as a sentence, paragraph, image, or code snippet. Embedding models map items into a vector space where items with related meanings often have vectors that are close under a chosen similarity measure.

Semantic search compares the embedding of a query with embeddings of stored content. Unlike keyword search, which primarily matches exact terms, semantic search can retrieve conceptually related passages even when the wording differs. It can still miss relevant information, so many systems combine semantic retrieval with keyword search, metadata filters, reranking, or other retrieval strategies.

Cosine similarity

Cosine similarity measures the cosine of the angle between two non-zero vectors. A common formula is: **cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)**. A larger value indicates a smaller angle between the vectors. The interpretation depends on the embedding model and retrieval setup; similarity is not a guarantee that a passage is factually correct.

5. Vector Databases

A vector database stores vectors and supports efficient similarity search. RAG applications commonly store an embedding for each document chunk together with the original text and metadata, such as source file, page, section, timestamp, or access permissions. At query time, the system searches for chunks whose vectors are similar to the query vector and returns the associated text.

Examples of vector storage and search technologies include:

- Chroma: an open-source database often used for local development and prototyping.
- FAISS: a library for efficient similarity search over dense vectors.
- Pinecone: a managed vector database service.
- Weaviate: a vector database with search and data-management features.
- Qdrant: a vector search engine and database.

6. What Is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation combines information retrieval with text generation. Rather than relying only on information encoded in model parameters, a RAG system retrieves relevant passages from a knowledge source and supplies them to a language model as context for an answer. This can make answers more grounded in a specific document collection and can help a system use private or frequently updated information.

A typical RAG pipeline

- **1. Ingest:** Load source files such as PDFs, text files, webpages, or office documents.

- **2. Extract:** Convert the source into text and retain useful metadata, including page numbers when available.
- **3. Clean:** Remove repeated headers, broken characters, and irrelevant boilerplate where appropriate.
- **4. Chunk:** Split long text into smaller, overlapping passages so each passage covers a coherent topic.
- **5. Embed:** Convert each chunk into a vector using an embedding model.
- **6. Index:** Store vectors, chunk text, and metadata in a vector database or search index.
- **7. Retrieve:** Embed the user's question and retrieve the most relevant passages.
- **8. Build context:** Select and arrange retrieved passages within the model's context budget.
- **9. Generate:** Ask an LLM to answer using the supplied context and to acknowledge missing evidence.
- **10. Evaluate:** Test retrieval quality, factual grounding, citation accuracy, latency, and cost.

7. Chunking Strategies

Chunking determines how a document is divided before indexing. Fixed-size chunking uses a target number of characters or tokens, often with overlap. Recursive chunking attempts to split at natural boundaries such as paragraphs and sentences. Structure-aware chunking uses headings, sections, tables, or page boundaries. Semantic chunking attempts to keep semantically related sentences together.

Chunks that are too small may omit necessary context. Chunks that are too large may contain unrelated content and consume more of the model's context window. A practical starting point is to experiment with a few hundred tokens per chunk and modest overlap, then evaluate against representative questions rather than assuming one setting is universally best.

8. Grounding, Citations, and Hallucination Reduction

Grounding means basing an answer on identifiable evidence, such as retrieved document passages. A well-designed RAG assistant should distinguish supported facts from inference, avoid inventing details absent from the source, and indicate when the retrieved material is insufficient. If possible, it should show source titles and page numbers or links so users can verify important claims.

- Use a clear instruction: answer from the supplied context and say when the answer is not present.
- Return source metadata with each retrieved chunk and preserve it through generation.
- Use reranking or hybrid search when basic vector retrieval performs poorly.
- Check whether each generated claim is supported by retrieved passages.
- Treat retrieved content as data, not as instructions that override system or developer rules.
- Protect private documents with access controls, retention policies, and appropriate logging.

9. Using Multiple LLM Providers

A RAG application can use one provider for generation and another for embeddings or fallback responses, but provider capabilities differ. For example, a developer may use Google's Gemini API or Groq-hosted models for generation while using a separate embedding model and Chroma for vector storage. Before implementation, check the current API documentation for supported models, authentication, rate limits, context limits, data handling, and pricing. Do not assume that every provider offers the same embedding capabilities.

Configuration tip: Keep API keys in environment variables or a secrets manager. Never place keys in a PDF, source repository, client-side code, or publicly accessible logs.

10. Evaluation of a RAG Application

A useful evaluation set contains realistic questions, expected answers, and the source passages that should support each answer. Consider these dimensions:

- Retrieval relevance: Are the retrieved chunks actually useful for the question?
- Context recall: Did retrieval find the evidence needed to answer?
- Faithfulness: Is the answer supported by the retrieved context?
- Answer relevance: Does the response directly address the question?
- Citation correctness: Do cited sources support the statements they accompany?
- Robustness: Does the system handle ambiguous, unanswerable, or adversarial questions safely?
- Operational performance: Measure latency, token use, cost, errors, and throughput.

11. Example Questions for Testing This PDF

- What is the difference between AI, machine learning, and deep learning?
- Explain the stages of a typical RAG pipeline.
- Why does a RAG system use embeddings?
- What information should be stored as metadata alongside a document chunk?
- How can a RAG application reduce unsupported answers?
- What are the trade-offs between small and large chunks?
- Why should API keys be stored in environment variables?
- Name three metrics or dimensions for evaluating a RAG system.

12. Glossary

AI: Artificial Intelligence; computing systems designed to perform tasks associated with intelligence.

ML: Machine Learning; methods that learn patterns from data.

DL: Deep Learning; machine learning based on multi-layer neural networks.

LLM: Large Language Model; a model trained to process and generate language.

Token: A unit of text processing used by language models; it may be a word, part of a word, or another text unit.

Embedding: A numerical vector representing an item in a learned space.

Chunk: A smaller passage created from a larger document for indexing and retrieval.

Retriever: A component that searches an indexed collection for relevant content.

Reranker: A component that reorders retrieved results using a relevance model or scoring method.

Vector database: A system that stores vectors and supports similarity search.

Grounding: Connecting generated claims to evidence from trusted or retrieved sources.

Hallucination: An output that sounds plausible but is inaccurate or unsupported.

Metadata: Descriptive information associated with a document or chunk, such as source and page number.

Conclusion

RAG systems combine retrieval and generation to answer questions using an external knowledge collection. Their quality depends on the full pipeline: document extraction, chunking, embeddings, indexing, retrieval, context

construction, generation, and evaluation. A strong implementation measures performance with real questions, keeps secrets secure, preserves source metadata, and communicates uncertainty when evidence is weak.

End of document — suitable as a starter knowledge source for RAG experimentation.